

面向个人研究的股票量化研究台设计与实现

——多源行情、向量化回测与 LLM 自主研究 Agent

王颢竣

2026 年 7 月 6 日

摘要

本文设计并实现了一个面向个人投资研究的股票量化研究台。系统以“看大盘、选个股、跑回测、AI 解读”为主线，采用前后端分离架构：后端使用 Django 与 Django REST Framework 提供 JSON 接口，前端基于 Vue 3 构建暗色金融风格的交互界面。针对免费行情数据源不稳定的问题，系统设计了多源降级与数据库缓存机制，实时指数与个股报价优先使用通达信协议直连，行业板块与涨跌广度使用东方财富接口并辅以短周期缓存。回测部分自行实现了向量化回测引擎，通过持仓序列的时间平移消除未来函数，支持双均线、RSI 与布林带三类策略，并输出累计收益、最大回撤、夏普比率与胜率等绩效指标。系统的核心创新在于引入了基于 Claude Agent SDK 的自主研究 Agent：Agent 在 Docker 临时沙箱中自主编写策略代码、运行回测、诊断结果并迭代改进，形成“假设—验证—诊断—改进”的研究闭环，最终产出可供参考的研究结论。数据库层面，系统从 SQLite 平滑迁移到 PostgreSQL 以支持并发与生产部署。实验表明，各功能模块均能在真实 A 股数据上正常运行，自主研究 Agent 能够在一次运行中完成十余次回测迭代并给出具备金融解释性的诚实结论。

关键词：量化研究；向量化回测；大语言模型；Agent；工具调用；沙箱执行；前后端分离

目录

第一章 绪论	4
1.1 选题背景	4
1.2 系统定位	4
1.3 主要工作	4
1.4 报告结构	5
第二章 系统总体设计	5
2.1 总体架构	5
2.2 技术选型	5
2.3 数据模型概览	6
第三章 行情数据采集与缓存层设计	6
3.1 问题分析	6
3.2 多源降级策略	6
3.3 缓存与容错机制	7
第四章 向量化回测引擎	8
4.1 设计思想	8
4.2 未来函数的消除	8
4.3 绩效指标	8
4.4 策略实现	9
第五章 智能体的设计与实现	9
5.1 量化对话 Agent	10
5.2 自主研究 Agent 的动机	10
5.3 整体流程	10

目录	3
5.4 Harness 与工具	10
5.5 沙箱与安全	12
5.6 异步执行与前端呈现	12
第六章 数据库设计与迁移	12
6.1 数据模型	12
6.2 从 SQLite 到 PostgreSQL 的迁移	13
第七章 运行结果与实验分析	14
7.1 功能运行结果	14
7.2 回测实验	14
7.3 自主研究 Agent 的运行分析	14
7.4 结果讨论	15
第八章 总结与展望	15
8.1 工作总结	15
8.2 不足与展望	15
第九章 项目总结与自评	15
9.1 项目总结	15
9.2 自我评价	16
第十章 开发中遇到的困难及解决方法	16
10.1 免费数据源的不稳定	16
10.2 依赖库的版本冲突	17
10.3 异步上下文中的数据库操作	17
10.4 代码执行的安全隔离	17

第1章 绪论

1.1 选题背景

近年来，量化投资在个人投资者中逐渐普及，但专业量化平台通常门槛较高，功能繁重且面向机构，个人投资者难以快速上手完成一次完整的研究流程。与此同时，免费的行情数据接口虽然可得，却普遍存在响应慢、连接不稳定的问题，数据采集往往成为个人量化研究的第一道障碍。此外，策略回测中的“未来函数”陷阱、绩效指标的正确理解、以及策略调参与结果解读，对缺乏经验的研究者并不友好。

大语言模型的发展为上述问题提供了新的思路。将模型的自然语言理解与代码生成能力，同量化回测引擎结合，有望降低“提出研究需求”的门槛，甚至让模型承担部分研究员的工作。本项目正是在这一背景下，尝试构建一个轻量、完整、以 AI 为核心亮点的个人量化研究台。

1.2 系统定位

本系统定位为“个人量化研究台”，其主线流程为：查看大盘行情、选择个股、运行策略回测、由 AI 进行智能解读。系统聚焦于研究、回测与智能分析，不涉及实盘交易。这一定位既保证了工程实现的可行性，也为论文提供了数据采集、回测引擎与 LLM Agent 三个层次的研究素材。

1.3 主要工作

本文的主要工作包括：

1. 设计并实现了多源降级的行情数据层，结合数据库缓存与逐源容错，提升了数据获取的稳定性；
2. 自行实现了向量化回测引擎，通过持仓时间平移消除未来函数，并实现了四项标准绩效指标；

3. 实现了基于工具调用的量化对话 Agent，可将自然语言需求转化为实际回测；
4. 在上述基础上进一步实现了自主研究 Agent：借助 Claude Agent SDK 与 Docker 沙箱，让模型自主编写并运行策略代码、迭代改进，形成研究闭环；
5. 完成了从 SQLite 到 PostgreSQL 的数据库迁移，并保留了本地零配置回落能力。

1.4 报告结构

本文其余章节安排如下：第二章介绍系统总体架构与技术选型；第三章介绍行情数据采集与缓存层设计；第四章介绍向量化回测引擎与绩效指标；第五章介绍量化对话 Agent 与自主研究 Agent 的设计与实现；第六章介绍数据库设计与迁移；第七章给出运行结果与实验分析；第八章总结全文并展望后续工作。

第2章 系统总体设计

2.1 总体架构

系统采用前后端分离架构。前端基于 Vue 3 与 Vite 构建单页应用，通过 HTTP 以 JSON 与后端通信，开发环境下由 Vite 代理转发以 /api 为前缀的请求。后端基于 Django 与 Django REST Framework，按业务划分为四个应用：行情 (market)、回测 (backtest)、智能体 (agent) 与账户 (accounts)。数据持久化在开发阶段使用 SQLite，生产阶段迁移至 PostgreSQL。

前端包含大盘看板、量化工作台、研究 Agent、回测历史、个股详情与登录六个主要视图；后端对外提供认证、行情、回测与智能体四组接口。各模块之间通过清晰的接口边界解耦，行情数据经缓存后供回测与智能体复用。

2.2 技术选型

系统的主要技术选型如表 2.1所示。选型的核心考量是：在保证功能完整的前提下，尽量使用免费、成熟且与研究台定位契合的技术栈。

表 2.1: 系统技术选型

层次	选型	说明
后端框架	Django + DRF	提供 JSON 接口
前端框架	Vue 3 + Vite + Pinia	暗色金融风界面
可视化	ECharts	K 线、热力图、收益曲线
组件库	PrimeVue	研究 Agent 界面
数据库	SQLite / PostgreSQL	开发 / 生产
行情数据源	通达信 (mootdx) + akshare	多源降级
回测引擎	自写 pandas 向量化	消除未来函数
智能体	Claude Agent SDK	自主研究闭环
沙箱	Docker 临时容器	隔离执行生成代码

2.3 数据模型概览

系统的核心数据实体包括：证券（Stock）与其日线行情（DailyBar）、策略（Strategy）与回测记录（Backtest）、用户自选股（WatchItem）、以及智能体的对话（Conversation）、消息（Message）与研究任务（ResearchTask）、研究迭代（ResearchIteration）。实体间关系将在第六章详细说明。

第 3 章 行情数据采集与缓存层设计

3.1 问题分析

免费行情数据源多以网页接口形式提供，存在响应慢、连接易中断的问题。若直接在每次请求时同步拉取外部数据，不仅用户体验差，还会因外部接口的偶发故障导致整页不可用。因此，数据层的设计目标是：在保证数据新鲜度的前提下，尽量减少对外部接口的直接依赖，并在单一数据源失效时保持系统整体可用。

3.2 多源降级策略

系统并未采用单一数据源，而是根据数据类型的不同特性，分别设计主源与备源，如表 3.1所示。

表 3.1: 分数据类型的多源降级策略

数据类型	主源	备源	说明
指数实时行情	通达信	akshare 日线	直连行情服务器，实时稳定
个股实时报价	通达信	——	供跑马灯与详情页
涨跌广度、板块	akshare	短周期缓存	数据字段最全
个股日线历史	akshare 新浪源	——	首次拉取后落库缓存

其中，实时行情之所以优先采用通达信协议，是因为其以原生二进制协议直连券商行情服务器，无需密钥，响应快且在本项目环境下稳定；而行业板块与涨跌广度的字段（如领涨股、换手率）在东方财富接口中最为完整，故予以保留，并通过缓存缓解其偶发波动。

3.3 缓存与容错机制

数据层实现了三项关键机制。其一是数据库缓存：个股日线数据首次拉取后写入数据库，后续请求命中缓存则不再访问外部接口，缓存覆盖判断允许一定的节假日空档容差。其二是短周期缓存与失败保留：行业板块数据设置约两分钟的有效期，在有效期内直接返回缓存；有效期过后若刷新失败，则继续返回上一次成功的数据，从而避免界面因外部波动而空白。其三是逐源容错：大盘看板接口对指数、广度、板块分别独立取数，任一数据源失败仅记入错误列表，不影响其余数据的正常返回。



图 3.1: 大盘看板：实时指数、涨跌广度与行业板块热力图

图 3.1展示了大盘看板的运行效果，其中指数卡片、涨跌广度与板块热力图分别由不同数据源经上述机制供给。

第4章 向量化回测引擎

4.1 设计思想

回测引擎是量化研究的核心。本系统未直接调用现成回测框架，而是基于 pandas 自行实现了一个向量化回测引擎，以便完整掌握其计算逻辑，也为论文提供实质的技术内容。所谓向量化，是指不采用逐日循环，而是以序列运算一次性计算出整条收益曲线，兼具简洁与效率。

4.2 未来函数的消除

回测可信度的关键在于避免“未来函数”，即在决策时使用了当时尚不可知的信息。本引擎的处理方式是：策略在第 T 日根据当日及此前的数据产生持仓信号，但该信号在第 $T+1$ 日才被兑现。实现上通过对持仓序列进行一次时间平移完成，如式 (4.1)所示：

$$r_t^{\text{strat}} = \text{position}_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} \quad (4.1)$$

其中 P_t 为第 t 日收盘价， position_{t-1} 为前一日确定的持仓（1 表示持有多头，0 表示空仓）。这一处理保证了“今天的信号、明天才成交”，从根本上杜绝了前视偏差。

4.3 绩效指标

引擎输出四项标准绩效指标，其定义与金融含义如下。设 E_t 为第 t 日的净值（初始为 1）， r_t^{strat} 为策略日收益。

- **累计收益**： $E_{\text{end}} - 1$ ，衡量整个区间的总体盈亏；
- **最大回撤**： $\min_t \left(\frac{E_t - \max_{s \leq t} E_s}{\max_{s \leq t} E_s} \right)$ ，衡量净值从历史峰值回落的最大幅度，是风险的重要度量；

- **夏普比率**: $\frac{\bar{r}^{\text{strat}}}{\sigma(r^{\text{strat}})}\sqrt{252}$, 以年化的方式衡量单位波动所获得的收益, 反映风险调整后的收益质量;
- **胜率**: 盈利交易日占有所有非空仓交易日的比例。

4.4 策略实现

系统实现了三类经典策略: 双均线策略在快速均线上穿慢速均线时持有; RSI 策略在超卖时买入、超买时卖出, 属于均值回归思路; 布林带策略在价格触及下轨时买入、回到中轨时卖出。三类策略均以统一的持仓序列形式输出, 交由引擎回溯。引擎还会检测持仓翻转点, 输出买卖标记以供前端在收益曲线上标注。



图 4.1: 量化工作台: 策略参数、K 线与收益曲线、买卖点标注

图 4.1展示了量化工作台的运行界面。以贵州茅台在 2023 至 2024 年的布林带策略为例, 回测得到约 +21.6% 的累计收益与 0.78 的夏普比率。

第 5 章 智能体的设计与实现

本章介绍系统的两类智能体: 面向对话的量化 Agent, 以及面向自主研究的研究 Agent。前者是基础, 后者是本项目的核心创新。

5.1 量化对话 Agent

量化对话 Agent 基于 Claude 的工具调用能力实现。后端维护一个循环，反复调用模型直至其结束回合；模型自主决定调用哪个工具、传入何种参数，后端执行工具后将结果返回，如此往复。Agent 注册了三个工具：列出可用策略、查询行情、运行回测。当用户以自然语言描述回测需求时，模型解析出股票代码、策略类型与参数，调用回测工具实际运行，并以中文解读收益、回撤、风险等结果。为保证可信，系统在系统提示中强制要求所有数字必须来自工具返回，不得编造。对话与工具调用记录均持久化保存，前端可展开查看调用轨迹。

5.2 自主研究 Agent 的动机

对话 Agent 虽然能将自然语言转化为回测，但它只能在预设的策略中选择参数，无法创造新的策略逻辑，也无法根据回测结果自我改进。为此，本项目进一步实现了自主研究 Agent，将其能力从“调用固定工具”提升为“在受控沙箱中自由编写并运行代码，并据结果迭代改进”，从而形成完整的研究闭环。

5.3 整体流程

自主研究 Agent 的工作流程如图 5.1 所示：用户提出研究任务后，Agent 先查看可用的量价数据与技术指标，随后编写策略代码并提交到 Docker 沙箱运行，读回真实的回测指标；若结果未达目标，则诊断问题（如回撤过大、信号过少、疑似过拟合）并改进代码，再次运行，如此迭代，直至收敛或达到迭代上限，最终产出研究结论。

5.4 Harness 与工具

研究 Agent 采用 Claude Agent SDK 作为驱动框架 (harness)。该框架提供了内建的智能体循环、工具编排与多轮上下文管理，避免了重复造轮子。系统以进程内 MCP 服务的形式注册工具：一个用于列出可用数据列，另一个（核心）用于在沙箱中执行策略代码并返回回测指标。认证与中转地址通过环境变量传入框架启动的子进程；同时通过白名单严格限定 Agent 只能调用上述沙箱工具，防止其

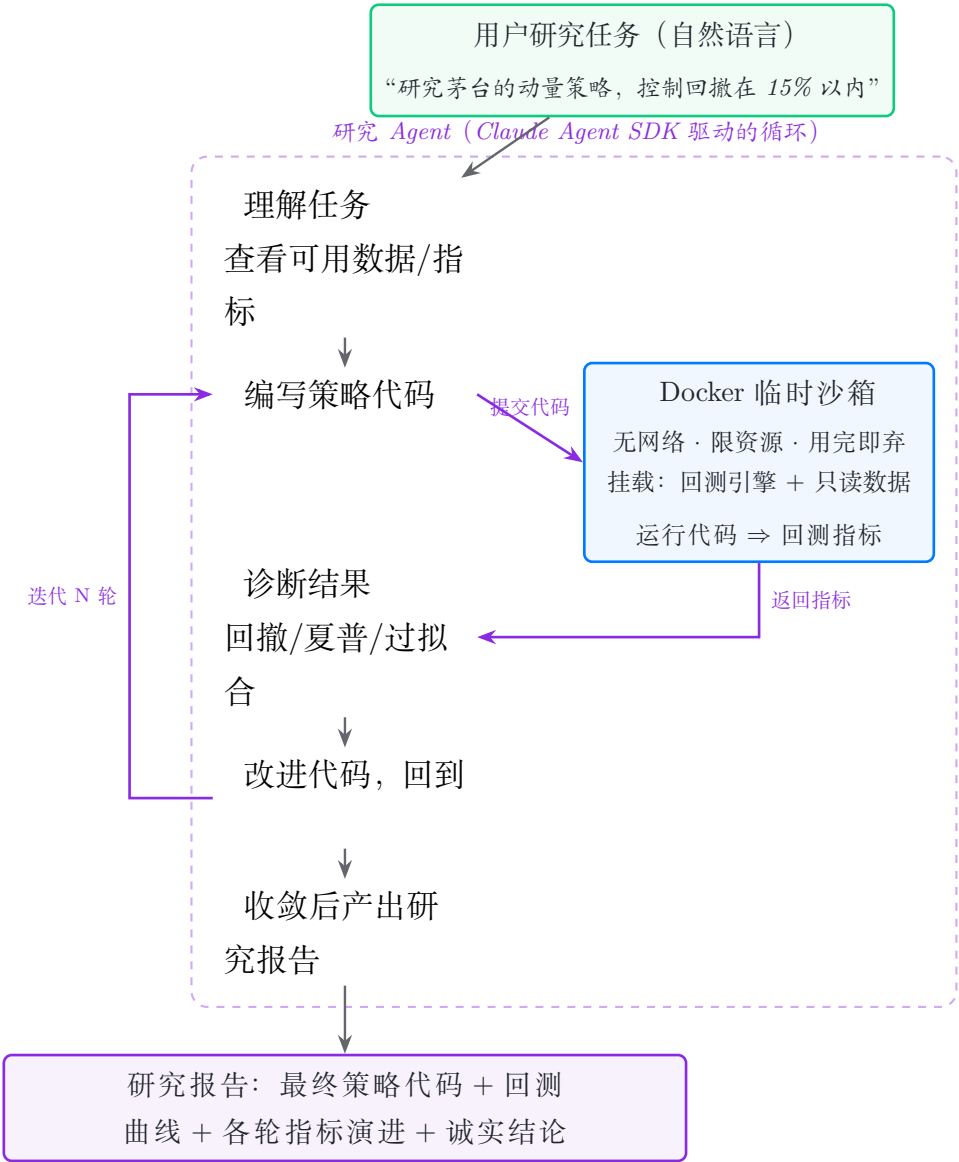


图 5.1: 自主研究 Agent 的研究闭环与沙箱执行架构

触及宿主系统的其他能力。

5.5 沙箱与安全

让模型自主执行其生成的代码是一项高风险操作，必须严格约束。系统为每次代码执行启动一个临时 Docker 容器，其安全措施如表 5.1 所示。

表 5.1: 沙箱执行的安全措施

风险	措施
代码外联、数据外泄	完全禁用网络
资源耗尽	限制内存、CPU、进程数并设执行超时
触及宿主文件系统	只读挂载引擎与数据，独立可写临时目录
提权	容器内以非特权用户运行
状态残留	一次一容器，用完即销毁

由于沙箱无网络，Agent 所需的数据必须在容器启动前从数据库拉取、预计算指标并物化为文件，再以只读方式挂入。沙箱内运行的回测引擎与主系统采用完全一致的计算逻辑，因此沙箱结果与系统回测结果可直接比较。

5.6 异步执行与前端呈现

一次研究通常需要数分钟，故采用异步方式执行：任务在后台线程中运行，每一步（思考、工具调用、代码运行及其指标）作为一条迭代记录写入数据库，前端通过轮询获取任务状态与迭代时间线并实时渲染。研究 Agent 的前端界面基于 PrimeVue 构建，如图 5.2 所示。

第 6 章 数据库设计与迁移

6.1 数据模型

系统的数据模型通过 Django ORM 定义，核心实体及其关系如下：用户与回测记录、自选股、对话为一对多关系；证券与其日线行情为一对多关系，并被回测记录与自选股引用；策略与回测记录为一对多关系；对话与消息为一对多关系；



图 5.2: 研究 Agent 界面：研究目标发起与迭代时间线

研究任务与研究迭代为一对多关系。回测指标、收益曲线、工具调用轨迹等结构灵活的数据，统一以 JSON 字段存储。实体关系如图 6.1所示。

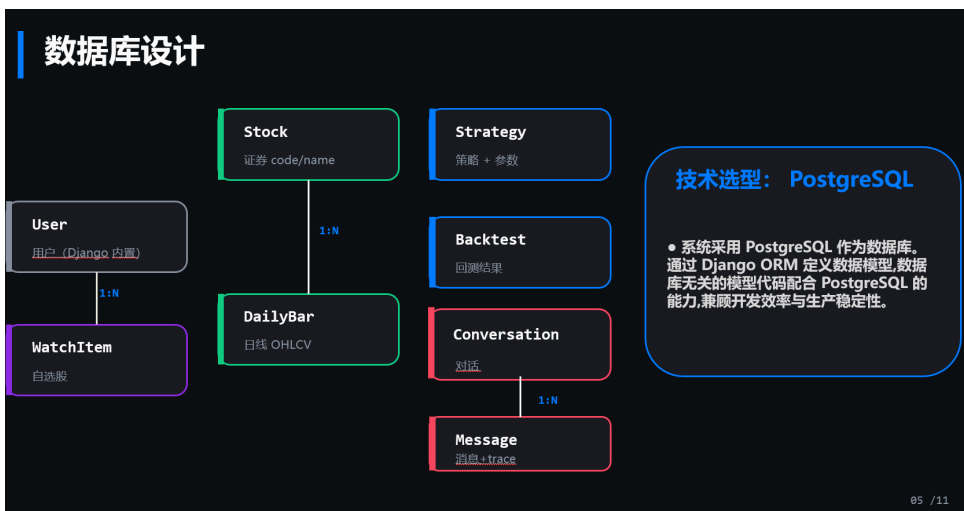


图 6.1: 数据库实体关系 (E-R) 图

6.2 从 SQLite 到 PostgreSQL 的迁移

开发阶段系统使用 SQLite，以零配置便于快速迭代。为支持并发写入与生产部署，系统迁移至 PostgreSQL。得益于 ORM 对底层数据库的抽象，迁移主要涉及数据库配置而非业务代码：系统通过环境变量判断，当配置了 PostgreSQL 连接时使用 PostgreSQL，否则回落至本地 SQLite，从而兼顾生产可用性与本地开发便利。PostgreSQL 以容器方式部署，其原生的 jsonb 类型也更适合存储系统中大量

使用的 JSON 字段。迁移过程通过数据导出与导入完成，实测各表记录数在迁移前后完全一致。

第 7 章 运行结果与实验分析

7.1 功能运行结果

系统各功能模块均在真实 A 股数据上完成了验证。大盘看板能够展示实时指数、涨跌广度与行业板块热力图；量化工作台能够对指定个股运行策略回测并绘制收益曲线与买卖点；用户系统支持注册、登录、自选股与回测历史。相关界面已在前文各图中展示。

7.2 回测实验

以贵州茅台的布林带策略为例，在 2023 至 2024 年区间内回测得到约 +21.6% 的累计收益与 0.78 的夏普比率，收益曲线与买卖点如图 4.1 所示，验证了回测引擎的正确性与可用性。

7.3 自主研究 Agent 的运行分析

为验证自主研究 Agent 的端到端能力，设置研究任务“用双均线策略研究贵州茅台，寻找夏普比率尽量高的均线参数组合”。Agent 在一次运行中自主完成了十余次沙箱回测迭代，其过程与结论具有明显的研究员特征。

Agent 的研究过程呈现出如下特征。首先，它测试了多组标准均线组合，发现其夏普比率均为负，并正确诊断出原因：该区间内标的整体处于下跌趋势，纯趋势跟随策略在下跌市中天然不利。其次，它尝试叠加 RSI 超买过滤，并观察到随着过滤阈值收紧，夏普比率单调改善（由约 -0.07 提升至约 0.30）；Agent 据此判断该改进来自有效逻辑而非偶然的参数过拟合。最后，它给出了最优组合，并诚实地指出了其局限：夏普绝对水平不高、换手率偏高、未做样本外验证，且趋势工具本身不适合该段行情。这一过程表明，Agent 不仅能自主编写与运行代码，还能对结果进行具备金融解释性的批判性分析，而非简单堆砌数字。

7.4 结果讨论

上述结果说明，将大语言模型的代码生成能力与受控沙箱、向量化回测引擎相结合，能够构成一个可运行、可复现的自主研究闭环。迭代过程中每一轮的代码与指标均被完整记录，这既为用户提供了透明的研究轨迹，也为进一步分析“Agent 的迭代是否真正改进了策略”提供了数据基础。

第8章 总结与展望

8.1 工作总结

本文设计并实现了一个面向个人研究的股票量化研究台，完成了多源降级的行情数据层、消除未来函数的向量化回测引擎、基于工具调用的量化对话 Agent，以及基于 Claude Agent SDK 与 Docker 沙箱的自主研究 Agent，并完成了数据库向 PostgreSQL 的迁移。系统在真实 A 股数据上验证了各模块的可用性，自主研究 Agent 能够完成多轮回测迭代并给出具备解释性的研究结论。

8.2 不足与展望

系统仍有若干可改进之处。其一，自主研究目前主要基于价量数据与技术指标，尚未接入外部因子库；后续可引入用户上传的研报知识库，并结合检索增强生成（RAG）在研究时提供参考，同时注重研报的时效性。其二，当前进度反馈采用轮询方式，后续可改为服务端推送以实现更实时的流式展示。其三，回测尚未支持交易成本与多标的组合，绩效评估也可引入样本外验证与参数寻优，以进一步提升研究结论的稳健性。

第9章 项目总结与自评

9.1 项目总结

回顾整个开发过程，本项目从一个“个人量化研究台”的构想出发，逐步落地为一个功能完整、层次清晰的系统。从行情数据的采集与缓存，到向量化回测引擎，再到两类智能体，最后完成数据库迁移与论文撰写，各阶段环环相扣。项目不

仅实现了预期的核心功能，更在自主研究 Agent 这一亮点上取得了超出最初设想的效果——它能够真正地自主编写代码、迭代改进，并给出具备金融解释性的诚实结论，而非停留在“聊天壳”层面。

9.2 自我评价

从完成度来看，系统覆盖了从数据、回测到智能分析的完整链路，各功能均在真实 A 股数据上通过了验证，具备可演示、可复现的特性。从工程质量来看，项目在多个环节体现了对稳定性与安全性的考量：数据层的多源降级与缓存容错、回测引擎对未来函数的严格消除、以及沙箱执行的多重安全约束，都不是为了功能而功能，而是针对真实问题做出的务实设计。

同时也应客观看待不足。系统目前仍以技术指标和价量数据为主，尚未接入更丰富的基本面因子与外部研报；回测未计入交易成本，绩效评估也缺少样本外验证；异步进度反馈采用轮询而非流式推送。这些都是在有限时间内做出的取舍，也是后续值得完善的方向。

总体而言，本项目较好地平衡了功能广度、工程深度与创新性，尤其是将大语言模型的代码生成能力与受控沙箱、量化回测相结合的自主研究闭环，具有一定的探索价值。

第 10 章 开发中遇到的困难及解决方法

在开发过程中遇到了若干具有代表性的困难，其分析与解决过程本身也构成了项目的重要收获，现择要说明如下。

10.1 免费数据源的不稳定

困难：项目初期直接使用免费行情接口，频繁遇到响应缓慢与连接中断，且不同接口在不同数据类型上的可靠性差异很大，单一数据源难以支撑稳定的看板展示。

解决：放弃“单一数据源”思路，转而按数据类型设计多源降级。实时行情改用通达信协议直连，稳定性显著提升；行业板块等字段最全的数据保留原接口，并辅以短周期缓存与“刷新失败保留上次结果”的策略；日线数据落库缓存，避免重复请求。同时对看板接口做逐源容错，使单一数据源的失效不再导致整页不可用。

10.2 依赖库的版本冲突

困难：引入通达信库时，其依赖将 HTTP 客户端库降级到较低版本；而后续引入智能体框架时，又要求较高版本，三个关键库之间存在潜在的版本冲突，稍有不慎便会破坏已有功能。

解决：逐一实测各库在目标版本下的实际表现，而非仅凭依赖声明判断。验证发现通达信库虽声明需要较低版本，但在较高版本下仍能正常拉取行情，最终确认三者可在统一版本下共存，避免了不必要的降级，也保证了各功能同时可用。

10.3 异步上下文中的数据库操作

困难：自主研究 Agent 的驱动框架内部采用异步事件循环，而记录研究迭代需要进行同步的数据库写入。二者相遇时，框架抛出“不能在异步上下文中执行同步数据库操作”的错误，导致研究任务无法记录进度。

解决：将数据库写入操作从事件循环所在线程剥离，交由一个独立的单线程执行器完成，事件循环仅负责提交写入请求并等待其结束。由于该执行器线程中不存在运行的事件循环，同步数据库操作得以正常进行；单线程也保证了迭代记录的写入顺序。

10.4 代码执行的安全隔离

困难：让大语言模型自主运行其生成的代码存在明显风险，若不加约束，可能出现资源耗尽、越权访问乃至数据外泄等问题。

解决：采用一次一销毁的临时 Docker 容器执行生成代码，并施加完全禁网、限制内存与 CPU、非特权用户运行、只读挂载依赖等多重约束，将风险控制在容器边界之内。由于沙箱无网络，所需数据在容器启动前预先物化并只读挂入，既满足隔离要求，又不影响回测运行。

参考文献

- [1] Django Software Foundation. Django Documentation.
<https://docs.djangoproject.com/>